

Array

- **Definition:**

An array is a fixed-size data structure that stores elements of the same type in contiguous memory locations.

It allows direct access to each element using an index. Arrays are one of the simplest and most commonly used data structures in programming.

- **Applications:**

- Storing student scores or marks in a class.
- Maintaining a list of items in a shopping cart.
- Storing pixel values in image processing.
- Used in databases for indexing and quick access.

- **Code in c :**

```
#include <stdio.h>
```

```
int main() {
```

```
    int myArray[5] = {10, 20, 30, 40, 50};
```

```
    printf("Array elements:\n");
```

```
    for(int i = 0; i < 5; i++) {
```

```
        printf("%d ", myArray[i]);
```

```
    }
```

```
    printf("\n");
```

```
    // Modify an element
```

```
myArray[1] = 25;
printf("Modified array:\n");
for(int i = 0; i < 5; i++) {
    printf("%d ", myArray[i]);
}
printf("\n");

return 0;
}
```

- **Reasoning and Examples:**

Arrays are useful when we know the number of elements in advance and need fast access using an index.

For example, in a student management system, an array can store scores of students so the system can quickly calculate the average, find the highest score, or sort the marks.

Arrays are also used in image processing where each pixel value is stored in a 2D array for easy manipulation.

Stack

- **Definition:**

A stack is a linear data structure that follows the Last In First Out (LIFO) principle.

This means the last element added to the stack is the first one to be removed.

Basic operations in a stack include push (add element), pop (remove element), and peek (view top element).

- **Applications:**

- Undo/Redo operations in text editors.
- Function calls in programming (call stack).
- Expression evaluation and syntax parsing.
- Browser history navigation.

- **Code in c:**

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int value)
```

```
{
```

```
    if(top == MAX - 1)
```

```
    {
```

```
        printf("Stack Overflow \n");
```

```
    }
```

```
    else
```

```
    {
```

```
        top++;
```

```
        stack[top] = value;
```

```
        printf("%d pushed into stack\n", value);
```

```
    }
```

```
}
```

```
void pop()
```

```
{
```

```
    if(top == -1)
    {
        printf("Stack Underflow \n");
    }
    else
    {
        printf("%d removed from stack\n", stack[top]);
        top--;
    }
}
```

```
void display()
{
    if(top == -1)
    {
        printf("Stack is empty\n");
    }
    else
    {
        printf("Stack elements:\n");
        for(int i = top; i >= 0; i--)
        {
            printf("%d\n", stack[i]);
        }
    }
}
```

```
int main()
{
    push(10);
```

```
    push(20) ;  
    push(30) ;  
  
    display() ;  
  
    pop() ;  
  
    display() ;  
  
    return 0 ;  
}
```

- **Characteristics:**

- Follows LIFO (Last In First Out) principle
- Operations at one end only (top)
- Fixed size if implemented with array
- Dynamic size possible if implemented with linked list

- **Why stacks are used:**

Stacks help manage data that must be processed in reverse order.

They allow efficient access to the most recent item, which is important for system operations and program logic.

- **In computer systems, stacks are used to:**

- Keep track of function calls (call stack)
- Manage memory efficiently
- Perform undo/redo operations
- Process expressions and syntax parsing